Lecture 7

Review: OOP & Collections
Inner Classes, Validation Constraints

OOP Review

Encapsulation

- Encapsulation is a fundamental principle of object-oriented programming (OOP) in Java.
- It involves bundling data (fields) and the methods that operate on that data together within a single unit called a class.
- This concept offers several key benefits:
  - Data Protection and Integrity
  - Improved Modularity and Maintainability
  - Increased Reusability

Inheritance

- Inheritance is a powerful mechanism in object-oriented programming (OOP) that allows you to **create new classes (subclasses) that inherit properties and behaviors from existing classes (superclasses).**
- This facilitates code reuse, promotes code organization, and helps model real-world relationships between entities.
- Key Concepts:
  - **Superclass (Parent Class):** The class from which features are inherited.
  - **Subclass (Child Class):** The class that inherits from the superclass.
  - **Inheritance Relationship:** Represented by the extends keyword in Java.

Benefits of Inheritance

- **Code Reusability**: Subclasses can inherit existing functionality from superclasses, avoiding code duplication and promoting efficiency.
- **Reduced Development Time**: Building upon existing code bases saves development time and effort.
- **Code Organization**: Inheritance creates a hierarchical structure, making code easier to understand and maintain.
- **Modeling Real-World Relationships**: Helps represent "is-a" relationships between objects, improving code understandability.

Overriding & Polymorphism

- Overriding and polymorphism are closely related concepts in object-oriented programming (OOP) that deal with the dynamic behavior of objects.

Overriding

- Allows a subclass to **redefine** a method inherited from its superclass.
- The redefined method has the *same name*, *parameter list*, and *return type* as the superclass method but provides its own implementation.
- The subclass method is called at runtime based on the actual object's type, even if a reference variable of the superclass type is used.

Polymorphism

- Literally means “many forms”.
- Refers to the ability of objects of different classes to respond differently to the same method call, thanks to overriding.
- This allows us to write code that works with objects of different types without knowing their exact concrete class until runtime.
- Promotes code flexibility and simplifies handling of related objects with similar behavior.

Abstract Class

- **Abstract classes** are a special type of class in Java that acts as a **partial blueprint for subclasses**. They cannot be instantiated directly (creating objects from them), but they can be **extended by subclasses** that inherit their behavior and implement any remaining incomplete parts.
- Key Characteristics:
  - Declared with the **abstract** keyword.
  - May contain both abstract and non-abstract methods.
  - Abstract methods:
    - Do not have a body (implementation).
    - Must be implemented by subclasses before they can be instantiated.
  - Non-abstract methods:
    - Provide default implementations that can be used or overridden by subclasses.

Benefits of abstract classes

- **Enforce common behavior**: Abstract methods ensure subclasses implement specific functionality, promoting consistency.
- **Code reusability:** Non-abstract methods and common functionality can be shared, reducing code duplication.
- **Polymorphism:** Abstract classes can be used in collections and method calls, allowing different implementations at runtime.

Interfaces

Interfaces are fundamental concepts in object-oriented programming (OOP) that **define contracts** for what a class must **be able to do** (its behavior) but **do not specify how it does it** (implementation details).

Benefits of Interfaces

- **Improved code organization**: Promotes separation of concerns, making code easier to understand and maintain.
- **Increased code flexibility**: Allows for multiple implementations (concrete classes) of the same interface, fostering modularity and adaptability.
- **Enhanced polymorphism**: Enables dynamic method dispatch at runtime based on the object's type, allowing for flexible handling of different implementations.
- **Decoupling**: Classes that implement an interface are not dependent on the concrete implementation details, facilitating looser coupling and reusability.

Java Collections Review

Collection

- Collection is a data structure used to hold multiple elements
  - Elements can be added, removed, or updated
  - Can support many other operations
- Two types: **List** and **Set**
  - **List**: values are accessed *by index* and a value can appear multiple times
  - **Set**: each value can appear only once, cannot be accessed by index

Inheritance hierarchy of Collection

- The entire collection framework is divided into four interfaces:
    1) **List**
    2) **Queue**
    3) **Set**
    4) **Map**
- `List`, `Queue` and `Set` inherit from `Collection` interface
- `Map` does not inherit from `Collection` interface

List

- It handles sequential list of objects
  - `ArrayList`, `Vector` and `LinkedList` classes implement this interface.
  - The `java.util.ArrayList` class is the most commonly used implementation of `List`

```
List<Integer> list = new ArrayList<Integer>();
list.add(1);
list.add(2);
list.add(2);
System.out.println(list.toString());
```

[1, 2, 2]

Queue

- A `Queue` handles a group of objects in which elements are removed only from the head. `LinkedList` and `PriorityQueue` classes implement this interface.

```
Queue<String> name = new LinkedList<>();

name.add("Emma");
name.add("Jessica");
name.add("Taylor Swift");
System.out.println("Initial queue: " + name.toString());

String firstElement = name.poll();
System.out.println("First element: " + firstElement);
System.out.println("Queue after retrieving the first element: " + name.toString());
```

Output:

```
Initial queue: [Emma, Jessica, Taylor Swift]
First element: Emma
Queue after retrieving the first element: [Jessica, Taylor Swift]
```

Set

- A `Set` handles a group of objects which must contain only unique elements. This interface is implemented by `HashSet` and `LinkedHashSet` classes and extended by `SortedSet` interface which in turn, is implemented by `TreeSet`.

```
Set<Integer> set = new HashSet<Integer>();
set.add(100);
set.add(200);
set.add(200);
System.out.println(set.toString());
```

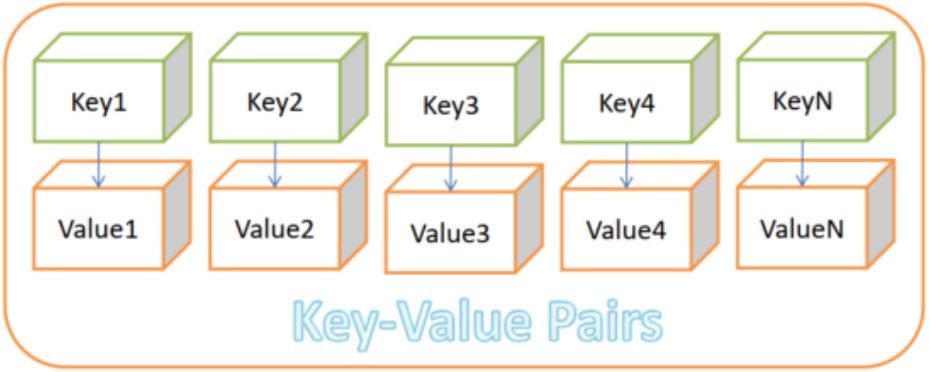
Output: **`[100, 200]`**

Map

- Handles a group of objects as key-value pairs.
  - Not inherited from `Collection` interface.
  - Implemented by `HashMap` and `HashTable` classes
  - Extended by `SortedMap` interface which in turn is implemented by `TreeMap`.

```
Map<String, Integer> map = new HashMap<>();

map.put("Alice", 25);
map.put("Bob", 30);
map.put("Charlie", 35);
System.out.println(map);
```



Output: `{Bob=30, Alice=25, Charlie=35}`

Sorting a set of objects

- To sort a set of objects, it is necessary to compare the objects.
- There are two ways to provide criteria for comparing objects:
  - Implement the `Comparable` interface and then write code to compare 2 objects in the `compareTo()` method (difficult to change the comparison criteria, rarely used).
  - Create an object from the `Comparator` interface then provide it to the `Collections.sort()` method (flexibility in comparison criteria, widely used).

Method #1: Sort a set of objects

```java
public class Student implements Comparable<Student> {
    public String fullName;
    public Double marks;

    public Student(String fullName, Double marks) {
        this.fullName = fullName;
        this.marks = marks;
    }

    @Override
    public int compareTo(Student o) {
        return marks.compareTo(o.marks);
    }
}
```

```java
List<Student> list = new ArrayList<>();
list.add(new Student("Emma", 5.0));
list.add(new Student("Jessica", 7.0));
list.add(new Student("Jennifer", 6.0));

Collections.sort(list);
```

Method #2: Sort a set of objects

```java
public class Student implements Comparable<Student> {
    public String fullName;
    public Double marks;

    public Student(String fullName, Double marks) {
        this.fullName = fullName;
        this.marks = marks;
    }

    @Override
    public int compareTo(Student o) {
        return marks.compareTo(o.marks);
    }
}
```

```java
List<Student> list = new ArrayList<>();
list.add(new Student("Emma", 5.0));
list.add(new Student("Jessica", 7.0));
list.add(new Student("Jennifer", 6.0));

Collections.sort(list);
```

Inner & Anonymous Classes

Inner Classes

- Defined **within another class** (outer class).
- Have access to the **outer class's members** (fields, methods).
- Can be of different types:
  - **Member inner class**: Defined within the body of the outer class but not within any method.
  - **Local inner class**: Defined within a method of the outer class.
  - **Anonymous inner class**: Defined and instantiated in a single statement, often used for one-time use cases.

Inner Classes: Example

```java
public class OuterClass {
    private int data = 10;

    public class MemberInnerClass {
        public void accessOuterData() {
            System.out.println("Inner class accessing outer data: " + data);
        }
    }

    public void someMethod() {
        class LocalInnerClass {
            public void useOuterMethod() {
                System.out.println("Local inner class using outer method");
            }
        }
        LocalInnerClass inner = new LocalInnerClass();
        inner.useOuterMethod();
    }
}
```

Benefits of Inner Classes

- **Improved code organization**: Group related functionality closer together, enhancing readability and maintainability.
- **Increased data encapsulation**: Inner classes can access and modify the outer class's private members, promoting data protection.

Anonymous Classes

- A special type of **local inner class** that **doesn't have a name.**
- Defined and **instantiated in a single statement**, often using an existing interface or abstract class as a blueprint.
- Useful for **short-lived objects** or when creating objects with specific behavior on the fly.

Anonymous Classes: Example #1

```java
// Define an interface
interface Greeter {
    void greet();
}

public class AnonymousClassSimple {
    public static void main(String[] args) {
        // An anonymous class that implements Greeter
        Greeter myGreeter = new Greeter() {
            @Override
            public void greet() {
                System.out.println("Anonymous class says hi");
            }
        };

        // Use the anonymous class
        myGreeter.greet();
    }
}
```

Anonymous Classes: Example #2

```java
public static void main(String[] args) {
    List<String> names = Arrays.asList("Carl", "Alice", "Bob");

    // Use anonymous class to define a custom Comparator
    Collections.sort(names, new Comparator<String>() {
        @Override
        public int compare(String s1, String s2) {
            // sort by string length
            return s1.length() - s2.length();
        }
    });

    System.out.println("Sorted by length: " + names);
}
```

Inner and Anonymous Classes

Feature	Inner Class	Anonymous Class
Definition	Can have a name and be defined separately	No name, defined and instantiated in a single statement
Access to outer class members	Yes	Yes, but only final or effectively final members
Reusability	Can be reused within the same scope	Not reusable, typically for one-time use

Validation Constraints

Conventional code

```java
public class CustomerSimple {
  private int id;
  private String name;
  public CustomerSimple(int id, String name) {
    this.id = id;
    this.name = name;
  }

  public int getId() {
    return id;
  }
  public String getName() {
    return name;
  }
  public void setName(String name) {
    this.name = name;
  }
}
```

More design details are needed...

- Can `id` be negative?
- Can `name` be uninitialized (i.e. takes `null`)?
- Should there be an operation to change value of `id`?
- What are the limit of `name`'s length?
- etc.

Domain constraints

- A **domain constraint** is a statement about what data values an attribute can take
- Some common domain constraints:
  - **mutable**: `true` | `false`
  - **optional**: `true` | `false`
  - **length** (for string type)): the max value length
  - **min** (for numeric type): min value
  - **max** (for numeric type): max value

Example of domain constraints

Attributes	type	mutable	optional	length	min	max
id	Integer	N	N	-	1	-
name	String	Y	N	30	-	-

Essential operations of an entity class

- At least include one constructor
  - [Java] default constructor may be omitted
- Getter (observer) methods
- Setter (mutator) methods for mutable attributes
- Data validation operations for attributes with domain constraints
- Default methods:
  - `toString()` method for String representation of object
  - `equals()` method for comparing objects

Default method: `toString()`

```
@Override
public String toString()
```

- No argument
  - Returns a string representation of an object

Default method: `equals()`

```
@Override
public boolean equals(Object o)
```

- Takes an `Object` argument and returns `boolean`
  - `true` if the argument is equal to the current object, `false` if otherwise
  - Also means the two objects are behaviorally equivalent

Data validation

- Validates input data against the domain constraints
  - invoked by constructor and setter operations
- Name: `validateX`, where `X` is an attribute name (first letter capitalized)
- Access modifier: `private`
- Parameters: match the attribute
- Return type: `boolean`
- May also invoke other validate operations

Data validation method examples

```java
private boolean validateId(int id) {
    if (id < 1) {
        return false;
    } else {
        return true;
    }
}

private boolean validateName(String name) {
    // check optional constraint
    if (name == null || name.length() == 0) {
        return false;
    }
    // check length constraint
    if (name.length() > 30) {
        return false;
    }
    return true;
}
```

Summary

- OOP Review
  - Encapsulation
  - Inheritance
  - Overriding & Polymorphism
  - Abstract Class
  - Interface
- Collection Review
  - List
  - Queue
  - Set
  - Map
- Inner class
- Anonymous class
- Validation
  - Constraints
  - Methods